PYTHON PROGRAMMING CASE STUDIES

Complete Assignment Record | 10 Case Studies

Each case study includes problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python source code, test cases, output screenshot, and conclusion.

Note: Rates, thresholds, and policies are illustrative assumptions because official values were not supplied. Replace them with prescribed values if provided.

1. University Scholarship Allocation

1. Problem Statement

Develop a program to evaluate applicants using aggregate marks, subject-wise minimum marks, attendance, and annual family income. Rank eligible applicants and resolve ties using predefined rules.

2. Objectives

- Check scholarship eligibility using all given criteria.
- Rank eligible students by aggregate marks, then attendance, then lower family income.
- Display selected students in rank order.

3. Assumptions

- Aggregate marks must be at least 75%.
- Every subject mark must be at least 60%.
- Attendance must be at least 75%.
- Annual family income must not exceed ₹250,000.
- Tie-break order: higher aggregate, higher attendance, lower income.

4. Algorithm

1. Start and read applicant details.
2. Validate that marks and attendance are between 0 and 100 and income is non-negative.
3. Check all four eligibility conditions.
4. Add eligible applicants to a list.
5. Sort by aggregate descending, attendance descending, and income ascending.
6. Display the ranked list and stop.

5. Flowchart

(START)
  ↓
Input applicant details
  ↓
Validate input
  ↓
Check aggregate ≥ 75, every subject ≥ 60,
attendance ≥ 75 and income ≤ 250000
  ↓
Eligible?
  ├── YES → Add to list
  └── NO → Reject
        ↓

Sort eligible list by aggregate ↓, attendance ↓, income ↑

↓

Display ranking

↓

(STOP)

Flowchart guide: draw Start/Stop as ovals, input/output as parallelograms, processing as rectangles, and decisions as diamonds.

6. Pseudocode

READ applicants\neligible = empty list\nFOR each applicant:\n IF aggregate >= 75 AND all subject marks >= 60\n AND attendance >= 75 AND income <= 250000:\n ADD applicant to eligible\nSORT eligible by (-aggregate, -attendance, income)\nDISPLAY eligible with rank

7. Python Source Code

```
applicants = [
    {"name": "Asha", "aggregate": 88, "subjects": [82, 90, 85],
     "attendance": 92, "income": 180000},
    {"name": "Bala", "aggregate": 88, "subjects": [75, 80, 60],
     "attendance": 95, "income": 200000},
    {"name": "Charan", "aggregate": 72, "subjects": [80, 75, 70],
     "attendance": 90, "income": 150000},
    {"name": "Divya", "aggregate": 81, "subjects": [78, 84, 80],
     "attendance": 85, "income": 220000},
]

eligible = []
for a in applicants:
    valid = (0 <= a["aggregate"] <= 100 and
             all(0 <= m <= 100 for m in a["subjects"]) and
             0 <= a["attendance"] <= 100 and a["income"] >= 0)
    if not valid:
        print("Invalid data:", a["name"])
        continue
    if (a["aggregate"] >= 75 and min(a["subjects"]) >= 60
            and a["attendance"] >= 75 and a["income"] <= 250000):
        eligible.append(a)

eligible.sort(key=lambda a: (-a["aggregate"], -a["attendance"], a["income"]))
print("Scholarship Merit List")
if not eligible:
    print("No eligible applicants")
else:
    for rank, a in enumerate(eligible, start=1):
        print(f"{rank}. {a['name']} - Aggregate: {a['aggregate']}%, "
              f"Attendance: {a['attendance']}%, Income: Rs.{a['income']}")
```

8. Test Cases

Input	Expected behavior	Expected result
Asha	88; subjects 82,90,85; attendance 92; income 180000	Eligible, rank 1
Bala	88; subjects 75,80,60; attendance 95; income 200000	Eligible; rank depends on the defined sort order (Bala ranks ahead of Asha because attendance is higher)
Charan	72; attendance 90; income 150000	Ineligible: aggregate below 75%
Divya	81; subjects 78,84,80; attendance 85; income 220000	Eligible, rank 3

9. Output Screenshot

```
Program Output

Scholarship Merit List
1. Bala - Aggregate: 88%, Attendance: 95%, Income: Rs.200000
2. Asha - Aggregate: 88%, Attendance: 92%, Income: Rs.180000
3. Divya - Aggregate: 81%, Attendance: 85%, Income: Rs.220000
```

10. Conclusion

The program checks scholarship conditions, excludes ineligible applicants, and produces a merit list using the specified tie-breaking rules.

2. Progressive Electricity Billing

1. Problem Statement

Calculate household electricity bills using consumption slabs, fixed charges, surcharges, and rebates. Compare monthly bills and flag unusual increases.

2. Objectives

- Calculate slab-wise energy charges.
- Add fixed charges and surcharge, then apply a qualifying rebate.
- Compare consecutive monthly bills and flag a rise greater than 30%.

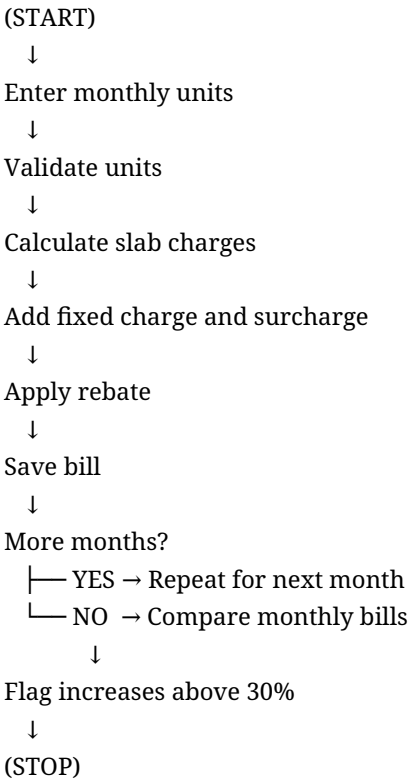
3. Assumptions

- 0–100 units: ₹1.50/unit; 101–200: ₹2.50/unit; above 200: ₹4/unit.
- Fixed charge is ₹50 per month.
- Surcharge is 5% of energy charge.
- If consumption is at most 100 units, rebate is ₹20.
- An increase greater than 30% over the previous bill is unusual.

4. Algorithm

7. Read monthly unit consumption.
8. Validate units are non-negative.
9. Calculate progressive slab charges.
10. Add fixed charge and surcharge; subtract any rebate.
11. Repeat for each month.
12. Compare each bill with the previous month's bill and flag increases above 30%.

5. Flowchart



Flowchart guide: draw Start/Stop as ovals, input/output as parallelograms, processing as rectangles, and decisions as diamonds.

6. Pseudocode

FOR each month:\n READ units\n energy = slab_charge(units)\n bill = energy + 50 + 0.05*energy\n IF units <= 100: bill = bill - 20\n SAVE bill\nFOR each bill after first:\n increase = (current - previous) / previous * 100\n IF increase > 30: print unusual increase

7. Python Source Code

```
def energy_charge(units):
    if units <= 100:
        return units * 1.50
    if units <= 200:
        return 100 * 1.50 + (units - 100) * 2.50
    return 100 * 1.50 + 100 * 2.50 + (units - 200) * 4.00

monthly_units = [80, 120, 260]
bills = []
for month, units in enumerate(monthly_units, start=1):
    if units < 0:
        raise ValueError("Units cannot be negative")
    energy = energy_charge(units)
    surcharge = energy * 0.05
    rebate = 20 if units <= 100 else 0
    total = energy + 50 + surcharge - rebate
    bills.append(total)
    print(f"Month {month}: {units} units, bill = Rs.{total:.2f}")

print("Monthly comparison")
for i in range(1, len(bills)):
    if bills[i - 1] == 0:
        increase = 0
    else:
        increase = (bills[i] - bills[i - 1]) / bills[i - 1] * 100
    status = "UNUSUAL INCREASE" if increase > 30 else "Normal"
    print(f"Month {i+1}: change = {increase:.1f}% - {status}")
```

8. Test Cases

Input	Expected behavior	Expected result
80 units	Energy ₹120; total ₹156	₹156.00
120 units	Energy ₹200; total ₹260	₹260.00
260 units	Energy ₹540; total ₹617	₹722.00
Negative units	Invalid input	ValueError

9. Output Screenshot

```
Program Output

Month 1: 80 units, bill = Rs.156.00
Month 2: 120 units, bill = Rs.260.00
Month 3: 260 units, bill = Rs.722.00
Monthly comparison
Month 2: change = 66.7% - UNUSUAL INCREASE
Month 3: change = 177.7% - UNUSUAL INCREASE
```

10. Conclusion

The program computes progressive electricity bills and identifies unusually large increases between consecutive months.

3. Hospital Billing and Patient Classification

1. Problem Statement

Process consultation, diagnostic, and treatment charges, apply conditional concessions, validate billing constraints, and classify patients according to a payment policy.

2. Objectives

- Calculate total hospital charges.
- Apply eligible concessions.
- Reject invalid or negative charges.
- Classify accounts by payment status.

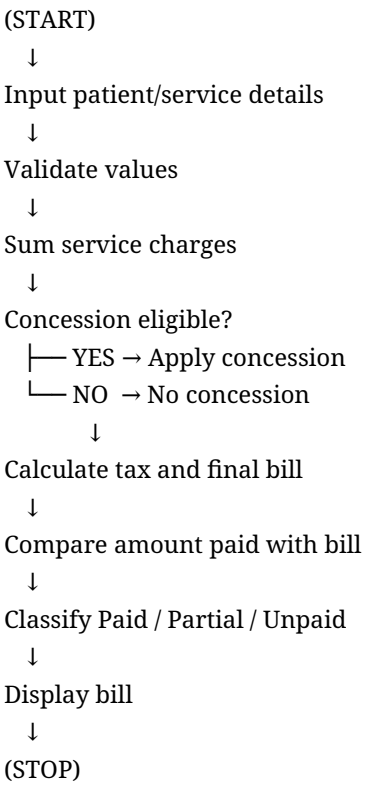
3. Assumptions

- Concession of 10% applies when the patient is a senior citizen (age 60 or above) or has a valid concession card.
- Concession is applied to the pre-tax service total.
- GST is assumed to be 5% of the discounted total for this exercise.
- Payment policy: Paid if amount paid covers final bill; Partial if some payment was made; Unpaid if no payment was made.

4. Algorithm

13. Read patient details and service charges.
14. Validate age, charges, and amount paid.
15. Calculate service total.
16. Apply 10% concession if eligible.
17. Calculate assumed tax and final bill.
18. Compare payment with bill and classify the patient account.
19. Display itemized bill and status.

5. Flowchart



Flowchart guide: draw Start/Stop as ovals, input/output as parallelograms, processing as rectangles, and decisions as diamonds.

6. Pseudocode

service_total = consultation + diagnostics + treatment\nIF age >= 60 OR valid_card: discount = 10% * service_total\nELSE discount = 0\nsubtotal = service_total - discount\ntax = 5% * subtotal\nfinal_bill = subtotal + tax\nIF paid >= final_bill: status = Paid\nELSE IF paid > 0: status = Partial\nELSE: status = Unpaid

7. Python Source Code

```
age = 65
consultation = 500
diagnostics = 1200
treatment = 3000
valid_card = False
amount_paid = 2500

charges = [consultation, diagnostics, treatment, amount_paid]
if age < 0 or any(x < 0 for x in charges):
    raise ValueError("Age and charges must be non-negative")

service_total = consultation + diagnostics + treatment
discount = service_total * 0.10 if age >= 60 or valid_card else 0
subtotal = service_total - discount
tax = subtotal * 0.05
final_bill = subtotal + tax

if amount_paid >= final_bill:
    status = "Paid"
elif amount_paid > 0:
    status = "Partial payment"
else:
    status = "Unpaid"

print("HOSPITAL BILL")
print(f"Service total: Rs.{service_total:.2f}")
print(f"Concession: Rs.{discount:.2f}")
print(f"Tax: Rs.{tax:.2f}")
print(f"Final bill: Rs.{final_bill:.2f}")
print(f"Amount paid: Rs.{amount_paid:.2f}")
print("Payment classification:", status)
```

8. Test Cases

Input	Expected behavior	Expected result
Age 65; charges 500,1200,3000; paid 2500	10% concession; partial payment	Final ₹4935.00; Partial payment
Age 30; no card; charges 100,200,300; paid 0	No concession; unpaid	Final ₹630.00; Unpaid
Negative treatment charge	Reject input	ValueError

9. Output Screenshot

```
Program Output

HOSPITAL BILL
Service total: Rs.4700.00
Concession: Rs.470.00
Tax: Rs.211.50
Final bill: Rs.4441.50
Amount paid: Rs.2500.00
Payment classification: Partial payment
```

10. Conclusion

The program prepares a hospital bill, applies conditional concessions, validates amounts, and classifies the payment status.

4. Employee Payroll and Deduction Analysis

1. Problem Statement

Calculate employee gross salary from basic pay, allowances, overtime, and incentives; apply progressive deductions and produce a formatted salary statement.

2. Objectives

- Compute allowances, overtime, incentives, and gross pay.
- Calculate progressive income deductions and fixed deductions.
- Display net salary in a formatted statement.

3. Assumptions

- HRA is 20% of basic pay; other allowance is 10% of basic pay.
- Overtime is paid at 1.5 times the hourly rate, with 160 regular hours per month.
- Incentive is supplied as an amount.
- Illustrative progressive deduction: 0% on first ₹20,000, 5% on next ₹20,000, 10% above ₹40,000.
- Provident fund deduction is 8% of basic pay.

4. Algorithm

20. Read basic pay, overtime hours, and incentive.
21. Validate all values are non-negative.
22. Calculate HRA, other allowance, and overtime pay.
23. Calculate gross salary.
24. Calculate progressive deduction and provident fund.
25. Subtract deductions from gross salary.
26. Print salary statement.

5. Flowchart

(START)
↓
Input salary details
↓
Validate
↓
Calculate allowances and overtime
↓
Calculate gross salary
↓
Calculate progressive deduction and PF
↓
Calculate net salary
↓
Print payslip
↓
(STOP)

Flowchart guide: draw Start/Stop as ovals, input/output as parallelograms, processing as rectangles, and decisions as diamonds.

6. Pseudocode

```
HRA = 0.20 * basic\nother = 0.10 * basic\novertime_pay = overtime_hours * (basic / 160) * 1.5\ngross = basic + HRA + other + overtime_pay + incentive\ntax = progressive_tax(gross)\nPF = 0.08 * basic\nnet = gross - tax - PF\nDISPLAY payslip
```

7. Python Source Code

```
basic = 30000
overtime_hours = 8
incentive = 2000

if basic < 0 or overtime_hours < 0 or incentive < 0:
    raise ValueError("Salary inputs must be non-negative")

hra = basic * 0.20
other_allowance = basic * 0.10
overtime_pay = overtime_hours * (basic / 160) * 1.5
gross = basic + hra + other_allowance + overtime_pay + incentive

def progressive_tax(amount):
    tax = 0
    tax += min(amount, 20000) * 0
    tax += min(max(amount - 20000, 0), 20000) * 0.05
    tax += max(amount - 40000, 0) * 0.10
    return tax

tax = progressive_tax(gross)
pf = basic * 0.08
net = gross - tax - pf

print("EMPLOYEE SALARY STATEMENT")
print(f"Basic pay:         Rs.{basic:,.2f}")
print(f"HRA:               Rs.{hra:,.2f}")
print(f"Other allowance:   Rs.{other_allowance:,.2f}")
print(f"Overtime pay:      Rs.{overtime_pay:,.2f}")
print(f"Incentive:         Rs.{incentive:,.2f}")
print(f"Gross salary:      Rs.{gross:,.2f}")
print(f"Progressive tax:   Rs.{tax:,.2f}")
print(f"PF deduction:      Rs.{pf:,.2f}")
print(f"Net salary:        Rs.{net:,.2f}")
```

8. Test Cases

Input	Expected behavior	Expected result
Basic ₹30000; OT 8 hours; incentive ₹2000	Calculate gross, tax, PF and net	Gross ₹43,250; tax ₹1,325; PF ₹2,400; net ₹39,525
Basic ₹0; OT 0; incentive ₹0	All calculations zero	Net ₹0.00
Negative basic pay	Reject input	ValueError

9. Output Screenshot

```
Program Output

EMPLOYEE SALARY STATEMENT
Basic pay:          Rs.30,000.00
HRA:                Rs.6,000.00
Other allowance:    Rs.3,000.00
Overtime pay:       Rs.2,250.00
Incentive:          Rs.2,000.00
Gross salary:       Rs.43,250.00
Progressive tax:    Rs.1,325.00
PF deduction:       Rs.2,400.00
Net salary:         Rs.39,525.00
```

10. Conclusion

The program calculates gross and net salary using allowances, overtime, incentives, and progressive deductions.

5. Multi-Plan Telecommunications Cost Optimization

1. Problem Statement

Compare mobile plans using expected usage, validity, and budget. Calculate excess usage charges and identify the most economical eligible plan.

2. Objectives

- Estimate the cost of each plan.
- Check validity and budget constraints.
- Include excess data or call charges.
- Choose the lowest-cost eligible plan.

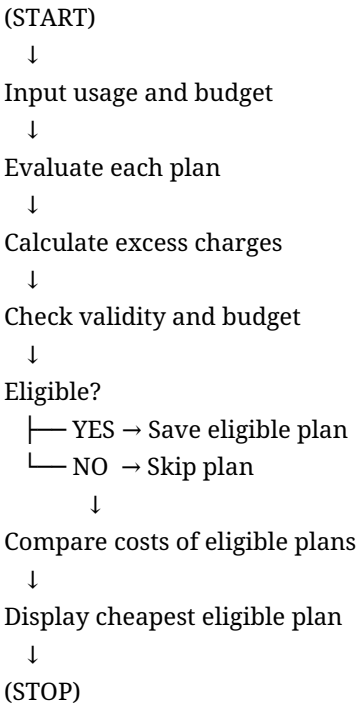
3. Assumptions

- Usage is measured in GB of data for a 30-day period.
- A plan is eligible only if validity is at least 30 days and total cost is within the budget.
- Excess data is charged per GB or fraction of a GB.
- Illustrative plans are defined in the program.

4. Algorithm

27. Read expected usage and budget.
28. For each plan, calculate excess data and extra charges.
29. Calculate total plan cost.
30. Check validity and budget.
31. Keep eligible plans and sort by total cost.
32. Display the lowest-cost plan or report no eligible plan.

5. Flowchart



Flowchart guide: draw Start/Stop as ovals, input/output as parallelograms, processing as rectangles, and decisions as diamonds.

6. Pseudocode

eligible = empty list\nFOR each plan:\n excess = max(usage - included_data, 0)\n extra = ceil(excess) * excess_rate\n total = plan_price + extra\n IF validity >= 30 AND total <= budget: add plan\nIF eligible: choose plan with minimum total\nELSE: print no eligible plan

7. Python Source Code

```
import math

usage_gb = 18
budget = 500
plans = [
   {"name": "Basic", "price": 199, "data": 10, "extra_rate": 20, "validity": 28},
   {"name": "Smart", "price": 349, "data": 20, "extra_rate": 15, "validity": 30},
   {"name": "Max", "price": 449, "data": 30, "extra_rate": 10, "validity": 56},
]

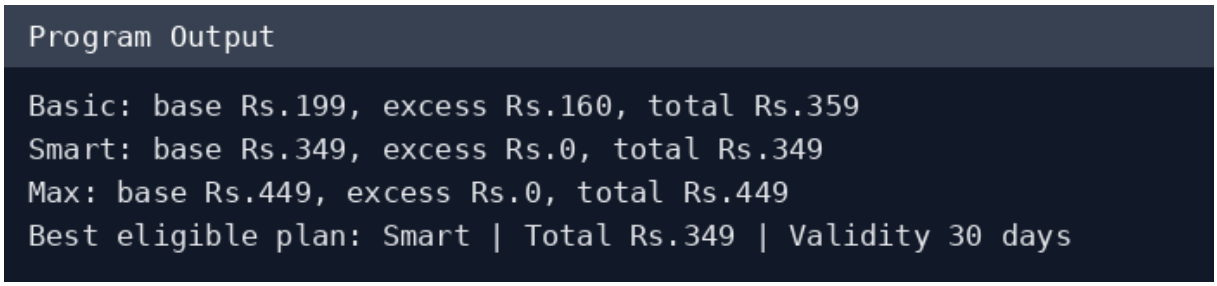
eligible = []
for p in plans:
   if usage_gb < 0 or budget < 0:
      raise ValueError("Usage and budget must be non-negative")
   excess = max(0, usage_gb - p["data"])
   extra_charge = math.ceil(excess) * p["extra_rate"]
   total = p["price"] + extra_charge
   print(f"{p['name']}: base Rs.{p['price']}, excess Rs.{extra_charge}, total Rs.{total}")
   if p["validity"] >= 30 and total <= budget:
      eligible.append((total, p["name"], p["validity"]))

if eligible:
   eligible.sort()
   total, name, validity = eligible[0]
   print(f"Best eligible plan: {name} | Total Rs.{total} | Validity {validity} days")
else:
   print("No plan satisfies validity and budget constraints.")
```

8. Test Cases

Input	Expected behavior	Expected result
Usage 18 GB; budget ₹500	Smart costs ₹349; Max ₹449; both eligible	Smart is cheapest eligible plan at ₹349
Usage 35 GB; budget ₹300	Plans exceed budget or incur excess charges	No eligible plan (depending on computed totals)
Negative usage	Reject input	ValueError

9. Output Screenshot



10. Conclusion

The program compares multiple mobile plans, accounts for excess usage, and selects the least expensive plan that meets constraints.

6. Railway Fare and Reservation Calculator

1. Problem Statement

Calculate fares for multiple passengers using distance slabs, travel class, age-based concessions, reservation fees, and cancellation penalties.

2. Objectives

- Calculate distance-based fares.
- Apply class multipliers and age concessions.
- Add reservation fees.
- Calculate cancellation refunds using the specified penalty.

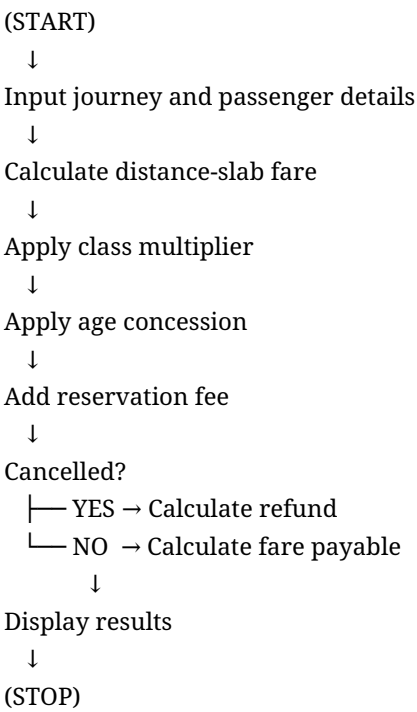
3. Assumptions

- Base fare is ₹0.75 per km for the first 100 km, ₹0.60 per km for the next 400 km, and ₹0.45 per km thereafter.
- Class multipliers: Sleeper 1.0, AC 2.0.
- Children under 5 travel free; passengers aged 60 or older receive a 20% concession.
- Reservation fee is ₹40 per passenger.
- Cancellation penalty is 25% of fare before reservation fee; reservation fee is non-refundable.

4. Algorithm

33. Read distance, class, passenger ages, and cancellation choice.
34. Validate distance and ages.
35. Calculate base fare by distance slabs.
36. Apply class multiplier and age concession.
37. Add reservation fee for each chargeable passenger.
38. If cancelled, calculate refund after cancellation penalty and non-refundable fee.
39. Display passenger-wise fares and total.

5. Flowchart



Flowchart guide: draw Start/Stop as ovals, input/output as parallelograms, processing as rectangles, and decisions as diamonds.

6. Pseudocode

base = distance_slab_fare(distance)\nfare = base * class_multiplier\nIF age < 5: fare = 0\nELSE IF age >= 60: fare = 0.80 * fare\nIF fare > 0: fare = fare + reservation_fee\nIF cancelled: refund = max(fare - 0.25*(fare-reservation_fee) - reservation_fee, 0)\nDISPLAY fare/refund

7. Python Source Code

```
def distance_fare(km):
    first = min(km, 100) * 0.75
    second = min(max(km - 100, 0), 400) * 0.60
    rest = max(km - 500, 0) * 0.45
    return first + second + rest

distance = 350
travel_class = "AC"
ages = [25, 62, 4]
cancelled = False
multiplier = {"Sleeper": 1.0, "AC": 2.0}

if distance <= 0 or travel_class not in multiplier or any(a < 0 for a in ages):
    raise ValueError("Invalid journey or passenger data")

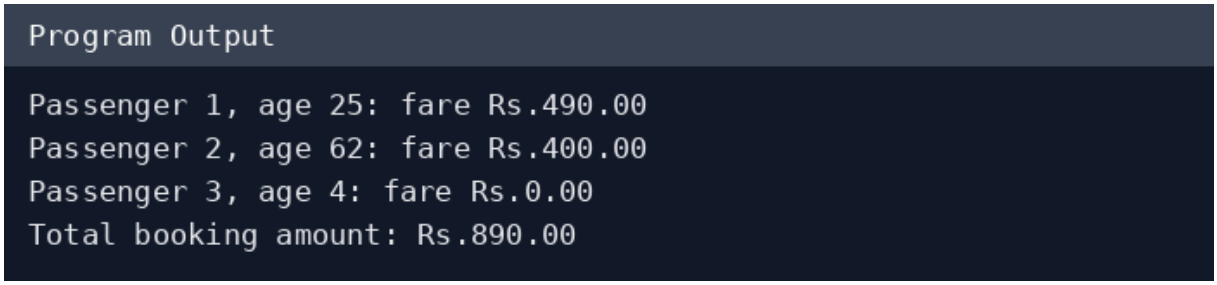
grand_total = 0
for i, age in enumerate(ages, start=1):
    fare = distance_fare(distance) * multiplier[travel_class]
    if age < 5:
        fare = 0
    elif age >= 60:
        fare *= 0.80
    reservation = 40 if fare > 0 else 0
    total = fare + reservation
    if cancelled and total > 0:
        refund = max(0, fare * 0.75)
        print(f"Passenger {i}: cancellation refund Rs.{refund:.2f}")
    else:
        grand_total += total
        print(f"Passenger {i}, age {age}: fare Rs.{total:.2f}")

if not cancelled:
    print(f"Total booking amount: Rs.{grand_total:.2f}")
```

8. Test Cases

Input	Expected behavior	Expected result
350 km; AC; ages 25, 62, 4	Calculate fares with concession and free travel under 5	Passenger fares displayed separately
Distance 0	Invalid journey	ValueError
Cancellation enabled	Apply cancellation penalty; reservation fee not refunded	Refund shown per passenger

9. Output Screenshot



10. Conclusion

The program calculates passenger-wise railway fares and applies class rates, age concessions, reservation fees, and cancellation rules.

7. Retail Billing with Multiple Discount Rules

1. Problem Statement

Process multiple products and quantities, apply category-specific taxes and threshold discounts, enforce a maximum concession, and generate an itemized invoice.

2. Objectives

- Calculate line totals for products.
- Apply category-specific tax rates.
- Apply a threshold discount subject to a maximum limit.
- Display an itemized invoice and final amount.

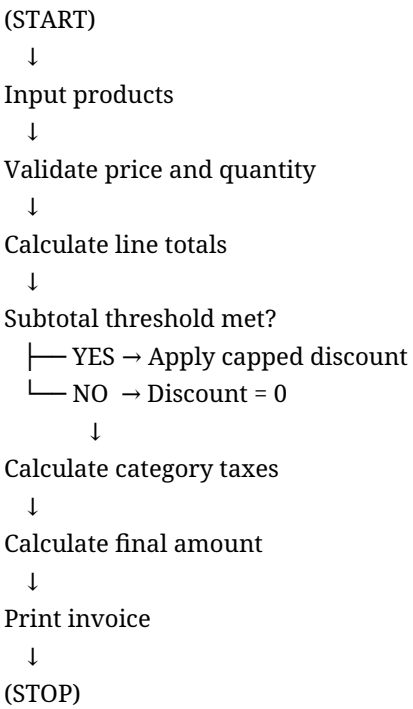
3. Assumptions

- Electronics tax is 18%; clothing tax is 5%; groceries tax is 0%.
- A 10% discount applies when the pre-tax subtotal is at least ₹5,000.
- Maximum discount is ₹1,000.
- Tax is calculated after discount, allocated proportionally across items for this example.

4. Algorithm

40. Read product name, category, price, and quantity.
41. Validate price and quantity.
42. Calculate each line amount and subtotal.
43. Apply discount if subtotal reaches ₹5,000, limited to ₹1,000.
44. Allocate discount proportionally and calculate category tax.
45. Display invoice totals.

5. Flowchart



Flowchart guide: draw Start/Stop as ovals, input/output as parallelograms, processing as rectangles, and decisions as diamonds.

6. Pseudocode

subtotal = sum(price * quantity)\nIF subtotal >= 5000: discount = min(10% * subtotal, 1000)\nELSE: discount = 0\nFOR each item:\n share = item_total / subtotal\n taxable = item_total - discount*share\n tax = taxable * category_tax_rate\nfinal = subtotal - discount + sum(taxes)\nDISPLAY itemized invoice

7. Python Source Code

```
products = [
    {"name": "Headphones", "category": "electronics", "price": 1800, "qty": 2},
    {"name": "Shirt", "category": "clothing", "price": 900, "qty": 2},
    {"name": "Rice", "category": "groceries", "price": 700, "qty": 1},
]
tax_rates = {"electronics": 0.18, "clothing": 0.05, "groceries": 0.00}

for p in products:
    if p["price"] < 0 or p["qty"] <= 0 or p["category"] not in tax_rates:
        raise ValueError("Invalid product details")

subtotal = sum(p["price"] * p["qty"] for p in products)
discount = min(subtotal * 0.10, 1000) if subtotal >= 5000 else 0
total_tax = 0

print("ITEMIZED INVOICE")
for p in products:
    line = p["price"] * p["qty"]
    share = line / subtotal if subtotal else 0
    taxable = line - discount * share
    tax = taxable * tax_rates[p["category"]]
    total_tax += tax
    print(f"{p['name']}: {p['qty']} x Rs.{p['price']} = Rs.{line:.2f}; tax Rs.{tax:.2f}")

final_total = subtotal - discount + total_tax
print(f"Subtotal: Rs.{subtotal:.2f}")
print(f"Discount: Rs.{discount:.2f}")
print(f"Tax: Rs.{total_tax:.2f}")
print(f"Final amount: Rs.{final_total:.2f}")
```

8. Test Cases

Input	Expected behavior	Expected result
Products subtotal ₹6,100	10% discount capped at ₹610 (below cap)	Discount ₹610
Subtotal ₹4,000	Threshold not reached	Discount ₹0
Quantity 0 or negative price	Reject input	ValueError

9. Output Screenshot

```
Program Output

ITEMIZED INVOICE
Headphones: 2 x Rs.1800 = Rs.3600.00; tax Rs.583.20
Shirt: 2 x Rs.900 = Rs.1800.00; tax Rs.81.00
Rice: 1 x Rs.700 = Rs.700.00; tax Rs.0.00
Subtotal: Rs.6100.00
Discount: Rs.610.00
Tax: Rs.664.20
Final amount: Rs.6154.20
```

10. Conclusion

The program generates an itemized retail invoice using quantity, category-based tax, threshold discounts, and a maximum discount limit.

8. Manufacturing Cost and Profitability Analysis

1. Problem Statement

Calculate material costs, labour, wastage, machine costs, and selling prices, then determine whether production meets a specified profit target.

2. Objectives

- Calculate material cost including wastage.
- Add labour and machine costs.
- Calculate total and per-unit cost.
- Evaluate profit and compare it with a target.

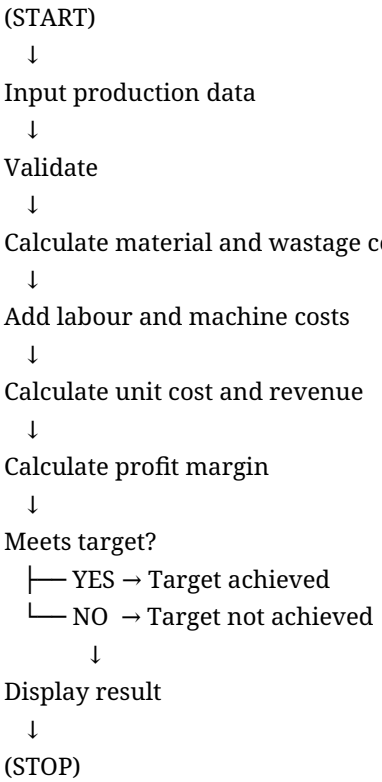
3. Assumptions

- Wastage is calculated as a percentage of raw material cost.
- Labour and machine costs are supplied as total batch costs.
- Selling price is per unit.
- Target profit margin is 20% of total production cost.

4. Algorithm

46. Read material quantity, unit price, wastage percentage, labour cost, machine cost, quantity produced, and selling price.
47. Validate all inputs.
48. Calculate raw material cost and wastage cost.
49. Calculate total production cost and unit cost.
50. Calculate revenue and profit margin.
51. Compare actual margin with target and display result.

5. Flowchart



Flowchart guide: draw Start/Stop as ovals, input/output as parallelograms, processing as rectangles, and decisions as diamonds.

6. Pseudocode

material = quantity * unit_price\nwastage = material * wastage_percent / 100\ntotal_cost = material + wastage + labour + machine\nunit_cost = total_cost / units_produced\nrevenue = units_produced * selling_price\nprofit = revenue - total_cost\nmargin = profit / total_cost * 100\nIF margin >= target: print target achieved

7. Python Source Code

```
material_qty = 500
unit_material_cost = 12
wastage_percent = 5
labour_cost = 2500
machine_cost = 1800
units_produced = 450
selling_price = 25
target_margin = 20

if (material_qty < 0 or unit_material_cost < 0 or labour_cost < 0
      or machine_cost < 0 or units_produced <= 0 or selling_price < 0
      or not 0 <= wastage_percent <= 100):
   raise ValueError("Invalid production data")

material_cost = material_qty * unit_material_cost
wastage_cost = material_cost * wastage_percent / 100
total_cost = material_cost + wastage_cost + labour_cost + machine_cost
unit_cost = total_cost / units_produced
revenue = units_produced * selling_price
profit = revenue - total_cost
margin = profit / total_cost * 100 if total_cost else 0

print("MANUFACTURING REPORT")
print(f"Material cost: Rs.{material_cost:.2f}")
print(f"Wastage cost: Rs.{wastage_cost:.2f}")
print(f"Labour cost: Rs.{labour_cost:.2f}")
print(f"Machine cost: Rs.{machine_cost:.2f}")
print(f"Total production cost: Rs.{total_cost:.2f}")
print(f"Cost per unit: Rs.{unit_cost:.2f}")
print(f"Revenue: Rs.{revenue:.2f}")
print(f"Profit: Rs.{profit:.2f}; Margin: {margin:.2f}%")
print("Target achieved" if margin >= target_margin else "Target not achieved")
```

8. Test Cases

Input	Expected behavior	Expected result
500 kg; ₹12/kg; 5% waste; labour ₹2500; machine ₹1800; 450 units; sell ₹25	Compute cost and margin	Total cost ₹10,600; revenue ₹11,250; profit ₹650; target not achieved
Units produced 0	Reject input	ValueError
Selling price high enough for 20% margin	Target achieved	Message: Target achieved

9. Output Screenshot

```
Program Output

MANUFACTURING REPORT
Material cost: Rs.6000.00
Wastage cost: Rs.300.00
Labour cost: Rs.2500.00
Machine cost: Rs.1800.00
Total production cost: Rs.10600.00
Cost per unit: Rs.23.56
Revenue: Rs.11250.00
Profit: Rs.650.00; Margin: 6.13%
Target not achieved
```

10. Conclusion

The program estimates manufacturing cost and profitability and checks whether the required profit margin is achieved.

9. Multi-Currency Travel Budget Analysis

1. Problem Statement

Convert travel expenses using supplied exchange rates, include conversion fees, compare planned spending with a budget, and identify the largest expense category.

2. Objectives

- Convert foreign-currency expenses into the home currency.
- Include conversion fees.
- Compare total spending with the budget.
- Identify the largest expense category.

3. Assumptions

- Home currency is INR.
- Exchange rates are supplied as INR per one unit of foreign currency.
- Conversion fee is 2% of converted amount.
- Budget and expenses must be non-negative.

4. Algorithm

52. Read travel expense categories, amounts, currencies, exchange rates, and budget.
53. Validate the supplied data.
54. Convert each expense to INR.
55. Add the conversion fee.
56. Calculate total spending and remaining budget or overspend.
57. Find the category with the largest converted expense.

5. Flowchart

(START)
↓
Input expenses, currencies, rates and budget
↓
Validate
↓
Convert each expense to INR
↓
Add conversion fees
↓
Sum expenses
↓
Compare with budget
↓
Find largest category
↓
Display report
↓
(STOP)

Flowchart guide: draw Start/Stop as ovals, input/output as parallelograms, processing as rectangles, and decisions as diamonds.

6. Pseudocode

total = 0\nFOR each expense:\n converted = amount * exchange_rate\n fee = converted * 0.02\n final = converted + fee\n SAVE final and add to total\nIF total <= budget: show remaining budget\nELSE: show overspend\nDISPLAY category with maximum final expense

7. Python Source Code

```
expenses = [
    {"category": "Accommodation", "amount": 300, "currency": "USD"},
    {"category": "Food", "amount": 120, "currency": "EUR"},
    {"category": "Transport", "amount": 10000, "currency": "INR"},
]
rates_to_inr = {"USD": 83.0, "EUR": 90.0, "INR": 1.0}
budget = 50000
fee_rate = 0.02
converted_expenses = []

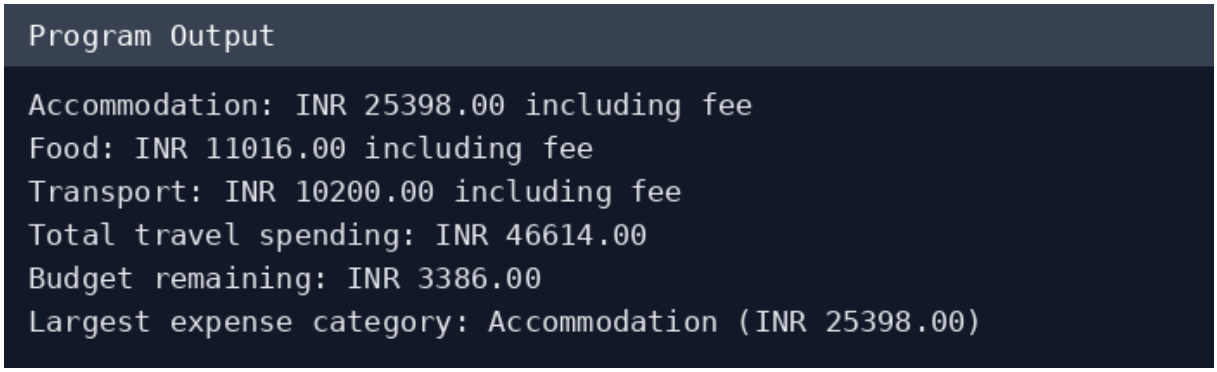
for e in expenses:
    if e["amount"] < 0 or e["currency"] not in rates_to_inr:
        raise ValueError("Invalid expense or missing exchange rate")
    converted = e["amount"] * rates_to_inr[e["currency"]]
    fee = converted * fee_rate
    final = converted + fee
    converted_expenses.append((e["category"], final))
    print(f"{e['category']}: INR {final:.2f} including fee")

total = sum(value for _, value in converted_expenses)
largest = max(converted_expenses, key=lambda item: item[1])
print(f"Total travel spending: INR {total:.2f}")
if total <= budget:
    print(f"Budget remaining: INR {budget-total:.2f}")
else:
    print(f"Budget exceeded by: INR {total-budget:.2f}")
print(f"Largest expense category: {largest[0]} (INR {largest[1]:.2f})")
```

8. Test Cases

Input	Expected behavior	Expected result
USD 300 at ₹83; EUR 120 at ₹90; INR 10000; budget ₹50000	Add 2% fee to each converted expense	Total ₹46,614; within ₹50,000 budget
Unknown currency code	Reject missing rate	ValueError
Budget lower than total	Report overspend	Overspend shown

9. Output Screenshot



10. Conclusion

The program converts travel expenses into INR, includes conversion fees, compares total spending with the budget, and identifies the largest category.

10. Campus Event Financial Planning

1. Problem Statement

Calculate event costs for venue, equipment, catering, transport, and certificates; determine participant contributions, sponsorship shortfalls, and the minimum registration count needed to break even.

2. Objectives

- Calculate total event expenditure.
- Calculate funds available from sponsorship and registration fees.
- Determine funding shortfall.
- Find the minimum number of registrations needed to break even.

3. Assumptions

- Venue, equipment, and transport are fixed costs.
- Catering and certificates are per-participant costs.
- Sponsorship is a fixed contribution.
- Registration fee is fixed per participant.
- Break-even registration count is rounded up to the next whole participant.

4. Algorithm

58. Read fixed costs, per-participant costs, sponsorship, and registration fee.
59. Validate all values.
60. Calculate fixed and variable costs.
61. Calculate total cost for expected registrations.
62. Calculate registration income and sponsorship total.
63. Determine funding surplus or shortfall.
64. Calculate minimum registrations needed to break even.

5. Flowchart

(START)
↓
Input fixed costs and per-person costs
↓
Validate
↓
Calculate fixed cost and variable cost
↓
Enter expected registrations
↓
Calculate total cost and available funds
↓
Determine shortfall or surplus
↓
Calculate break-even registrations
↓
Display event budget
↓
(STOP)

Flowchart guide: draw Start/Stop as ovals, input/output as parallelograms, processing as rectangles, and decisions as diamonds.

6. Pseudocode

fixed = venue + equipment + transport\nvariable_per_person = catering + certificate\ntotal_cost = fixed + expected_people * variable_per_person\nfunds = sponsorship + expected_people * registration_fee\nshortfall = max(total_cost - funds, 0)\ncontribution_per_person = variable_per_person + fixed/expected_people (if expected_people > 0)\nnet_per_registration = registration_fee - variable_per_person\nIF net_per_registration <= 0: break_even not possible\nELSE break_even = ceil((fixed - sponsorship) / net_per_registration), minimum 0

7. Python Source Code

```
import math

venue = 12000
equipment = 5000
transport = 3000
catering_per_person = 180
certificate_per_person = 20
sponsorship = 10000
registration_fee = 300
expected_participants = 80

values = [venue, equipment, transport, catering_per_person,
        certificate_per_person, sponsorship, registration_fee,
        expected_participants]
if any(v < 0 for v in values):
    raise ValueError("All costs and counts must be non-negative")

fixed_cost = venue + equipment + transport
variable_per_person = catering_per_person + certificate_per_person
total_cost = fixed_cost + expected_participants * variable_per_person
registration_income = expected_participants * registration_fee
available_funds = registration_income + sponsorship
shortfall = max(total_cost - available_funds, 0)
surplus = max(available_funds - total_cost, 0)

net_per_registration = registration_fee - variable_per_person
if fixed_cost <= sponsorship:
    break_even = 0
elif net_per_registration <= 0:
    break_even = None
else:
    break_even = math.ceil((fixed_cost - sponsorship) / net_per_registration)

print("CAMPUS EVENT BUDGET")
print(f"Fixed costs: Rs.{fixed_cost:.2f}")
print(f"Variable cost per participant: Rs.{variable_per_person:.2f}")
print(f"Total cost for {expected_participants} participants: Rs.{total_cost:.2f}")
print(f"Registration income: Rs.{registration_income:.2f}")
print(f"Sponsorship: Rs.{sponsorship:.2f}")
print(f"Funding shortfall: Rs.{shortfall:.2f}")
print(f"Surplus: Rs.{surplus:.2f}")
print("Break-even registrations:", break_even if break_even is not None else "Not possible at this fee")
```

8. Test Cases

Input	Expected behavior	Expected result
80 participants; fixed costs ₹20000; variable ₹200 each; sponsorship ₹10000; fee ₹300	Total cost ₹36000; funds ₹34000	Shortfall ₹2,000; break-even 100 participants
Negative venue cost	Reject input	ValueError
Registration fee <= variable cost and	No finite break-even count	Not possible at this fee

sponsorship below fixed cost

9. Output Screenshot

```
Program Output

CAMPUS EVENT BUDGET
Fixed costs: Rs.20000.00
Variable cost per participant: Rs.200.00
Total cost for 80 participants: Rs.36000.00
Registration income: Rs.24000.00
Sponsorship: Rs.10000.00
Funding shortfall: Rs.2000.00
Surplus: Rs.0.00
Break-even registrations: 100
```

10. Conclusion

The program prepares a campus event budget, estimates sponsorship shortfalls, and calculates the minimum registrations needed to break even.